

5.4 — Manage Context Effectively in Large Codebase Exploration

Concept

Claude's context window is a finite resource. In large codebase exploration, it is the most likely bottleneck you will hit — not capability, not tool availability. Understanding how context degrades and how to counter it is the difference between a session that delivers answers and one that drifts into noise.

Domain mental model: Think of the context window as working memory. Humans can hold roughly seven items in working memory at once; beyond that, earlier items get displaced. Claude's context works similarly. As you fill it with file contents, earlier instructions and findings become statistically less influential. The model does not forget them entirely, but their weight in generation drops. This is context degradation.

Context Degradation

When you read many files sequentially in a single session, two things happen. First, the raw token count climbs toward the limit, reducing the space available for new information. Second, and more insidiously, the signal-to-noise ratio falls. Early findings that were precise and well-grounded become buried under later file dumps. Instructions given at the start of the conversation carry less influence when they are thousands of tokens behind the current generation point.

Symptoms of context degradation include: repeated observations that were already made earlier, loss of earlier constraints (e.g., the model stops following a naming convention it acknowledged at the start), and reduced precision in answers as the session grows longer.

Scratchpad Files

The solution to degradation is externalization. Instead of relying on the context window to hold findings, write them to a persistent scratchpad file as you go. After each meaningful discovery, append a structured note: the file path, what was found, and why it matters. When `/compact` runs or the session is handed off, the scratchpad survives.

The scratchpad serves two purposes: it gives you a durable record independent of context state, and it gives Claude a single dense source to re-read at the start of a new session or after compaction, rather than re-reading dozens of source files.

A scratchpad entry might look like:

```
## auth/middleware.ts
- JWT validation lives here, not in the route handlers
- Token expiry checked at line 47; no refresh logic present
- Dependency: relies on `config/secrets.ts` for the signing key
```

The `/compact` Command

`/compact` instructs Claude Code to summarize the current conversation and replace the verbose history with a condensed version, freeing up context space. Use it proactively — before the window fills — rather than reactively when quality has already degraded. A good rule of thumb: run `/compact` after completing a discrete investigation phase (e.g., after mapping the auth layer, before exploring the data access layer).

After `/compact`, re-anchor the session by reading your scratchpad file and stating the current objective explicitly. This restores the working state with minimal token cost.

Subagent Delegation

When a codebase has multiple independent subsystems to explore, delegate each slice to a subagent. Each subagent starts with a fresh context window containing only its assigned scope. This is more efficient than one agent reading everything: parallel exploration, no cross-contamination of context, and each agent returns a focused summary rather than a sprawling transcript.

Coordinator pattern: the orchestrating agent defines the exploration boundaries, launches subagents with scoped instructions, and collects their summaries into the shared scratchpad. The coordinator itself never reads the raw files — only the summaries.

Incremental Exploration Strategy

Resist the impulse to read everything. Large codebases contain far more irrelevant code than relevant code for any given question. Use a funnel:

1. **Glob** — find files matching a pattern (e.g., `**/*.service.ts`) to understand the shape of the codebase
2. **Grep** — search for the specific symbol, function, or pattern you need
3. **Read** — open only the files that grep identified as containing the target

This three-stage funnel means you spend context tokens only on files that have demonstrated relevance. A session that reads 8 targeted files instead of 80 files retains far more capacity for reasoning.

Code Example

Scenario: You need to understand how a large Node.js API handles rate limiting. The repository has 200+ files.

Step 1 — Glob to map the terrain

```
Glob pattern: **/*.middleware.ts
Result: 6 files found – auth, cors, logging, rate-limit, timeout, validation
```

You now know rate limiting is isolated to a single middleware file. No need to read the others.

Step 2 — Grep to find the implementation entry point

```
Grep: "rateLimiter" in *.ts
Matches: rate-limit.middleware.ts (definition), app.ts (registration)
```

Two files. Read only those.

Step 3 — Read targeted files and write to scratchpad

```
Read: rate-limit.middleware.ts
Finding: Uses sliding window algorithm, Redis-backed, limit configurable via env var RATE_LIMIT

Scratchpad entry written:
## rate-limit.middleware.ts
- Sliding window, Redis store
- Limit: process.env.RATE_LIMIT_MAX (default 100)
- Applied globally in app.ts line 34
- No per-route override mechanism present
```

Step 4 — /compact before moving to next subsystem

```
/compact
→ Session summarized. Token count reset to ~15% of window.

Re-anchor: "Read scratchpad at /tmp/exploration-notes.md.
Next objective: understand the database connection pooling strategy."
```

Total files read: 2. Context used for actual reasoning: high. Context wasted on irrelevant code: zero.

Common Mistakes

- 1. Reading every file "just to be thorough"** This is the most common failure mode. Reading files that do not contain the answer displaces the context that does. Start with Grep to earn the right to read a file.
- 2. Relying on the context window as the only memory** Findings that exist only in the conversation transcript are one `/compact` away from being lost. Write to a scratchpad immediately after discovering something meaningful.
- 3. Running /compact reactively** By the time you notice degradation, quality has already dropped. Run `/compact` at natural phase boundaries — after completing exploration of one subsystem, before starting another.
- 4. Launching subagents without scoped instructions** A subagent given "explore the codebase" will behave no differently than a single agent with a full context. Scope subagents to a specific directory, layer, or question. The value of delegation comes from the boundaries.
- 5. Not re-anchoring after /compact** After compaction, Claude's condensed summary may omit specific constraints or objectives stated earlier. Always re-read your scratchpad and restate the current goal explicitly. Treat post-compact re-anchoring as mandatory, not optional.
- 6. Confusing file count with coverage** Reading 50 files does not mean you have covered 50% of the codebase's behavior. Behavior is concentrated in a small fraction of files. Invest context tokens in the high-

signal files identified by Grep, not in a breadth-first sweep.

References

- Anthropic documentation: Context window management and `/compact` behavior
- Exam domain 5: Context and Memory Management in Agent Systems
- Related handouts: 1.2 (Coordinator-Subagent Patterns), 1.6 (Task Decomposition), 5.1 (Memory Types)
- See also: Claude Code built-in tools — Glob, Grep, and Read form the canonical incremental exploration stack